数据科学中的计算机技能第二次作业

韦承成

2026001425

Keywords: RcppArmadillo

1 图像卷积

1.1 实现效果:

实现 “镜像边界（mirror padding)” 的 2D 图像卷积，分别使用纯 R 语言循环实现、RcppArmadillo C++ 实现。镜像填充（mirror /reflection padding）指的是当卷积核滑动到图像边缘，超出原图坐标时，将图像边缘向外镜像反射取值，而不是补 0。

1.2 实现说明

1.2.1 卷积原理

设输入图像矩阵 $\boldsymbol{I_{H*W}}$，尺寸 $H \times W$；卷积核 $\boldsymbol{K}$，尺寸 $k_h \times k_w$（一般为奇数，如 3×3)。对于输出图像每个位置 (i, j):

$$O_{i,j} = \sum_a \sum_b K_{a,b} I_{Mirror}(i + a, j + b)$$

I_{mirror} 代表镜像边界采样函数，坐标超出原图范围时镜像反射.

1.2.2 实现思路

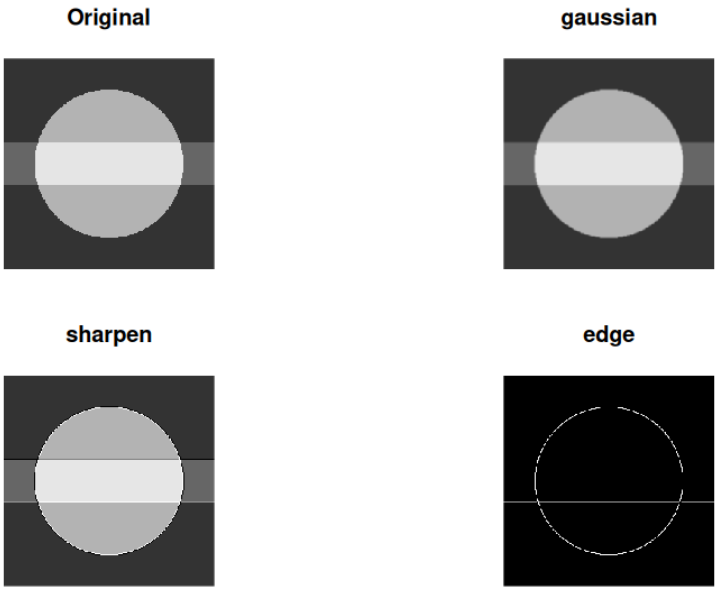
遍历输出图像的每一个像素位置;
遍历卷积核的每一个元素;
计算原图采样坐标，使用镜像规则修正越界坐标;
累加乘积，得到输出像素值。
区别是 C++ 中数组下标从 0 开始 R 语言中 matrix 下标从 1 开始.

1.3 运行结果

1.3.1 运行时间

	method	seconds	repetitions	rows	cols
1	C++	0.0044	10	256	256
2	R	0.9950	10	256	256

1.3.2 卷积效果 (R)



1.3.3 结果差异

```
> source("~/homework2/convolution/image-convolution.R")
$gaussian_differences
[1] 0

$sharpen_differences
[1] 0

$edge_differences
[1] 0
```

这个结果表明，三种卷积核下 Rcpp 与 R 语言的运行结果没有差异。

2 多元正态分布的对数概率密度

2.1 实现效果

实现多元正态分布的对数概率密度计算函数，分别编写纯 R 语言版本与基于 RcppArmadillo 的 C++ 版本；验证两种实现的数值一致性，检验计算结果是否等价；对比两个版本的运行耗时，分析 C++ 实现带来的计算加速效果；理解多元正态对数概率密度的数学形式，体会采用对数变换与 Cholesky 分解规避数值下溢、提升数值稳定性的原理。

2.2 实现说明

2.2.1 计算对数密度

由于

$$f(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{p/2}\det(\boldsymbol{\Sigma})^{1/2}} \exp\left\{-\frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})\right\}$$

所以

$$\log f(\boldsymbol{x};\boldsymbol{\mu},\boldsymbol{\Sigma}) = -\frac{p}{2}\log(2\pi) - \frac{1}{2}\log\det(\boldsymbol{\Sigma}) - \frac{1}{2}(\boldsymbol{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\boldsymbol{x}-\boldsymbol{\mu})$$

直接计算原始密度会出现极小浮点数，极易发生数值下溢，因此计算中普遍使用对数形式。

2.2.2 算法实现

1. 计算残差向量 $\boldsymbol{r} = \boldsymbol{x} - \boldsymbol{\mu}$；
2. 对协方差矩阵 $\boldsymbol{\Sigma}$ 进行 Cholesky 分解 $\boldsymbol{\Sigma} = LL^\top$，$L$ 为下三角矩阵；
3. 求解 $\boldsymbol{\Sigma}^{-1}\boldsymbol{r}$：先解 $L\boldsymbol{z} = \boldsymbol{r}$，再解 $L^\top\boldsymbol{w} = \boldsymbol{z}$，得到 $\boldsymbol{w} = \boldsymbol{\Sigma}^{-1}\boldsymbol{r}$；
4. 计算二次型项：$\boldsymbol{r}^\top\boldsymbol{\Sigma}^{-1}\boldsymbol{r} = \boldsymbol{r}^\top\boldsymbol{w}$；
5. 计算协方差矩阵行列式对数：$\log|\boldsymbol{\Sigma}| = 2\sum_{i=1}^{p}\log(L_{ii})$；
6. 代入对数密度公式，计算最终的多元正态对数概率密度。

选用 Cholesky 分解而不是直接求逆矩阵的原因在于一是数值稳定性更好；二是计算效率高于直接矩阵求逆；三是正定矩阵一定可以 Cholesky 分解。

2.3 运行结果

```
> source("~/homework2/mvn_log_density/mvn-log-density.R")
   actual  expected
-1.504404 -1.504404
mvn-log-likelihood计算耗时： 0.01043797 s
   actual  expected
-5.079362 -5.079362
mvn-log-likelihood计算耗时： 0.0007247925 s
   actual  expected
-4.096459 -4.096459
mvn-log-likelihood计算耗时： 0.0001790524 s
   actual  expected
-230.0917 -230.0917
mvn-log-likelihood计算耗时： 0.0001654625 s
```

从这张图片可以看出使用 Cholesky 分解计算对数概率密度与真实值没有差异，算法运行时间如图。

3 Kalman Smoothing

3.1 预期效果

先进行前向 Kalman filtering，得到后向计算所需的预测均值、预测协方差和过滤结果；再进行后向 smoothing；最后一个时刻的平滑均值应等于该时刻的过滤均值，并绘图展示真实路径、观测值、过滤路径和平滑路径。

3.2 模型原理

线性高斯状态空间模型定义如下：

$$\begin{cases} \boldsymbol{x}_t = F\boldsymbol{x}_{t-1} + \boldsymbol{w}_t, \quad \boldsymbol{w}_t \sim \mathcal{N}(0, Q) \\ \boldsymbol{y}_t = H\boldsymbol{x}_t + \boldsymbol{v}_t, \quad \boldsymbol{v}_t \sim \mathcal{N}(0, R) \end{cases}$$

其中：

- $\boldsymbol{x}_t$：t 时刻的隐状态向量；
- $\boldsymbol{y}_t$：t 时刻的观测向量；
- F：状态转移矩阵；
- H：观测矩阵；
- Q：状态噪声的协方差矩阵；
- R：观测噪声的协方差矩阵。

卡尔曼滤波为前向递推算法，依次使用时刻 $1, 2, \ldots, T$ 的观测，得到滤波估计 $\hat{\boldsymbol{x}}_{t|t}$ 和滤波协方差 $P_{t|t}$，该估计仅利用 $1, \ldots, t$ 的观测信息。

1. 预测步：

$$\begin{aligned} \hat{\boldsymbol{x}}_{t|t-1} &= F\hat{\boldsymbol{x}}_{t-1|t-1} \\ P_{t|t-1} &= FP_{t-1|t-1}F^\top + Q \end{aligned}$$

2. 更新步：

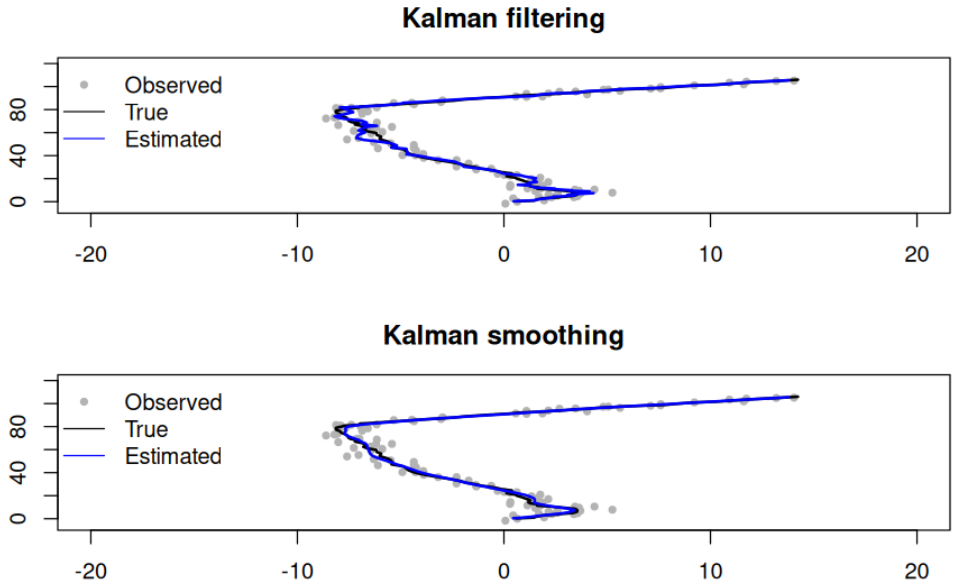
$$\begin{aligned} S_t &= HP_{t|t-1}H^\top + R \\ K_t &= P_{t|t-1}H^\top S_t^{-1} \\ \hat{\boldsymbol{x}}_{t|t} &= \hat{\boldsymbol{x}}_{t|t-1} + K_t\left(\boldsymbol{y}_t - H\hat{\boldsymbol{x}}_{t|t-1}\right) \\ P_{t|t} &= \left(I - K_tH\right)P_{t|t-1} \end{aligned}$$

卡尔曼平滑 (Kalman Smoothing) 是在卡尔曼滤波完成之后，从末尾时刻 T 反向递推至时刻 1，利用全部观测 $\{y_1, y_2, \ldots, y_T\}$ 得到平滑估计 $\hat{\boldsymbol{x}}_{t|T}$ 与平滑协方差 $P_{t|T}$。

$$\begin{aligned} C_t &= P_{t|t}F^\top P_{t+1|t}^{-1} \\ \hat{\boldsymbol{x}}_{t|T} &= \hat{\boldsymbol{x}}_{t|t} + C_t\left(\hat{\boldsymbol{x}}_{t+1|T} - F\hat{\boldsymbol{x}}_{t|t}\right) \\ P_{t|T} &= P_{t|t} + C_t\left(P_{t+1|T} - P_{t+1|t}\right)C_t^\top \end{aligned}$$

边界条件：$\hat{\boldsymbol{x}}_{T|T} = \hat{\boldsymbol{x}}_{T|T}$，$P_{T|T} = P_{T|T}$。

3.3 运行结果



从这张图片可以看出，Kalman Smoothing 后的曲线更加光滑，更加贴近真实值，这一点从两种位置估计的均方误差也能看出。

```
> source("~/homework2/Kalman-smoothing/kalman-smoothing.R")
filtering_mse smoothing_mse
   0.25161834    0.06381335
kalman-smoothing 计算耗时 0.009931803 s
```

可以看出 Kalman Smoothing 的均方误差更小.

解释 filtering 和 smoothing 用到的信息不相同:

卡尔曼滤波（Filtering）估计时刻 t 的状态，仅采用 1 到 t 的观测，只用到当前与历史信息，支持在线实时估计。

卡尔曼平滑（Smoothing）估计时刻 t 的状态，使用全部 1 到 T 的观测结果，包含 t 之后未来的观测信息，属于离线估计，估计精度更高。

4 Ridge regression 与 Tikhonov regularization

4.1 问题描述

在线性回归问题中，给定设计矩阵 $(X \in \mathbb{R}^{n\times p})$ 和观测向量 $(y \in \mathbb{R}^n)$，希望求得参数向量 $(\beta \in \mathbb{R}^p)$，使模型预测值 $X\beta$ 尽可能接近观测数据 y。

经典最小二乘方法通过求解

$$\min_{\beta\in\mathbb{R}^p} ||X\beta - y||_2^2$$

估计参数。然而，当矩阵 X 存在严重的病态性，或者各特征之间存在较强的多重共线性时，最小二乘解可能对数据扰动十分敏感，导致估计结果不稳定。

为改善数值稳定性并控制模型复杂度，可以引入正则化方法。本部分主要研究 Ridge Regression（岭回归）和 Tikhonov Regularization（Tikhonov 正则化），推导相应的求解公式，并讨论正则化参数的选择方法。

4.2 Ridge Regression(岭回归)

4.2.1 数学模型

岭回归在最小二乘目标函数中加入参数向量的二范数惩罚项，其优化问题为

$$\min_{\beta\in\mathbb{R}^p} ||X\beta - y||_2^2 + \lambda||\beta||_2^2$$

其中，$\lambda \geq 0$ 为正则化参数。

当 $\lambda = 0$ 时，该问题退化为经典最小二乘问题；当 $\lambda > 0$ 时，惩罚项限制参数向量的大小，从而降低估计结果对数据扰动的敏感程度。

4.2.2 求解过程

记目标函数为

$$J(\beta) = ||X\beta - y||_2^2 + \lambda||\beta||_2^2$$

将目标函数展开，得到

$$J(\beta) = (X\beta - y)^T(X\beta - y) + \lambda\beta^T\beta$$

对 β 求梯度：

$$\nabla J(\beta) = 2X^T(X\beta - y) + 2\lambda\beta$$

令梯度等于零，可得

$$X^TX\beta - X^Ty + \lambda\beta = 0$$

因此，

$$(X^TX + \lambda I)\beta = X^Ty$$

当 $(\lambda > 0)$ 时，矩阵 $(X^TX + \lambda I)$ 是对称正定矩阵，因而可逆。于是岭回归的解析解为 $\hat{\beta}_\lambda = (X^TX + \lambda I)^{-1}X^Ty$

该公式表明，正则化相当于在正规方程的系数矩阵中加入 λI，从而改善矩阵的条件数，提高求解的稳定性。实际计算时，通常不显式计算矩阵的逆，而是直接求解线性方程组

$$(X^TX + \lambda I)\hat{\beta}_\lambda = X^Ty$$

, 以减少计算开销并改善数值稳定性。

4.3 Tikhonov Regularization

4.3.1 数学模型

Tikhonov 正则化是岭回归的一般化形式，其优化问题为

$$\min_{\beta \in \mathbb{R}^p} ||X\beta - y||_2^2 + \lambda ||B\beta||_2^2$$

,

其中，B^{p*p} 为给定的可逆正则化矩阵，$\lambda \geq 0$ 为正则化参数。

与岭回归相比，Tikhonov 正则化并不一定直接惩罚参数向量 β 的整体大小，而是通过矩阵 B 对参数施加特定的约束。例如, 当 $B = I_p$ 时，Tikhonov 正则化退化为岭回归。

4.3.2 求解过程

记目标函数为

$$J(\beta) = ||X\beta - y||_2^2 + \lambda ||B\beta||_2^2$$

.

利用矩阵求梯度公式，可得

$$\nabla J(\beta) = 2X^T(X\beta - y) + 2\lambda B^T B\beta$$

.

令梯度等于零，得到

$$X^T X\beta - X^T y + \lambda B^T B\beta = 0$$

.

整理可得 Tikhonov 正则化对应的正规方程:

$$(X^T X + \lambda B^T B)\beta = X^T y$$

.

当矩阵 $X^T X + \lambda L^T L$ 可逆时，其解为

$$\hat{\beta}_\lambda = (X^T X + \lambda B^T B)^{-1} X^T y$$

.

需要注意的是，当 $\lambda > 0$ 时，该矩阵并不一定总是可逆。其可逆的充要条件为

$$\mathrm{Null}(X) \cap \mathrm{Null}(B) = \{\mathbf{0}\}$$

其中 Null(·) 表示矩阵的零空间。

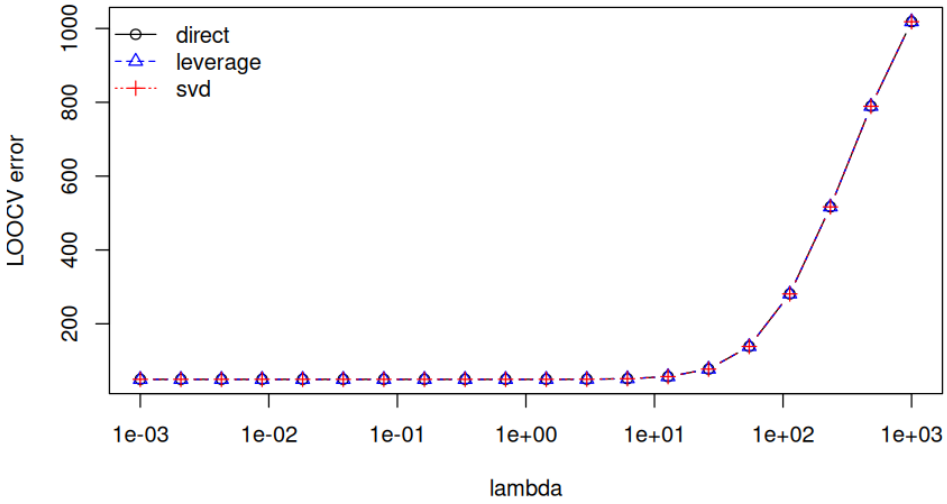
因此，Tikhonov 正则化可以通过选择合适的矩阵 (B)，在保证拟合效果的同时，对解施加符合实际问题需求的结构约束。

4.4 运行结果

4.4.1 Ridge Resgression 运行结果

```
> source("~/homework2/Ridge Regression and Tikhonov Regression/ridge-cv.R")
direct
actual =  4.196365
expected =  4.196365
diff =  0
leverage
actual =  4.196365
expected =  4.196365
diff =  2.664535e-15
svd
actual =  4.196365
expected =  4.196365
diff =  8.881784e-16
          method max_absolute_error selected_lambda
direct     direct       0.000000e+00       0.6951928
leverage leverage       1.421085e-13       0.6951928
svd           svd       1.705303e-12       0.6951928
          method median_seconds repetitions calls_per_batch   n  p lambda_count
direct     direct       0.060000           5               1 200 20           20
leverage leverage       0.231170           5             100 200 20           20
svd           svd       0.000176           5             500 200 20           20
```

从这张图片看出，三种方法运行结果是一致的，但三种方法运行时间差异明显：对于直接进行岭回归，对每个 λ，逐个删除 200 个样本并重新拟合，共进行 $20 \times 200 = 4000$ 次拟合；对于杠杆法，每个 λ 只需全样本拟合一次，但仍需计算 leverage。当前实现还需要求解矩阵方程来计算所有 leverage，因此开销较大；而 SVD 分解法只进行一次 SVD 分解，之后对所有 λ 可以重复使用分解结果，避免重复求解矩阵分解.



从上图看出，最大绝对误差都很小，选取的最优 λ 大约为 0.695.

4.4.2 Tikhonov Regularization 运行结果

```
> source("~/homework2/Ridge Regression and Tikhonov Regularization/tikhonov.R")
coefficient_error      fitted_error consistency_error
     2.220446e-15      7.105427e-15      6.217249e-15
ridge_max_absolute_error
           4.440892e-15
```

从图片看出，Tikhonov Regularization 的最大绝对误差很小，且运行结果表明 $B = I_p$ 时，Tikhonov Regularization 与 ridge regression 的系数估计一致.